RL and Behavior Tree Model for Pacman Capture the Flag

GROUP13
Banah Abdeljaber Gawtam Chithra Ramesh
17-11-2003 12-10-2001
banaha@kth.se gawtam@kth.se

Abstract

In this project, we address the problem of autonomous decision-making in a competitive multi-agent environment using a Unity3D implementation of PacMan Capture the Flag. This game highlights AI challenges such as partial observability, cooperative/adversarial play, and dynamic adaptation—relevant to domains like robotic team coordination and autonomous surveillance. To solve this, we developed a hybrid system that combines a custom behavior tree for high-level strategy selection with reinforcement learning (RL) models trained for three behaviors: attacking (collecting food), defending (chasing opponents), and escaping (avoiding threats). RL was chosen for its ability to learn adaptive policies in dynamic, partially observable settings. This combination offers strategic structure through the behavior tree and tactical flexibility through learned actions, enabling agents to switch strategies based on game state. Our contributions include the design of the behavior tree and the specialized RL models. We also discuss the challenges and limitations of relying heavily on RL for complex multi-agent coordination. This work is significant because it addresses core challenges in autonomous systems, including reasoning under partial knowledge, adapting to dynamic environments, and coordinating multiple agents in real time—challenges that mirror real-world robotic applications. Our results offer insights into the difficulties of combining learned and rule-based strategies, highlighting directions for future exploration in multi-agent decision-making.

1 Introduction

In this project, agents must autonomously play a 3D Unity implementation of PacMan Capture the Flag. The environment consists of a symmetric maze divided into two sides: red and blue. Teams spawn on their own side as ghosts and must cross into the opponent's side to transform into pacman, allowing them to collect food pills. Successfully returning to their own side drops the collected pills near the border, contributing to their team's score. Teams must balance offense (collecting pills) and defense (preventing opponents from collecting pills), while navigating additional dynamics such as super-pills, agent elimination, and limited sensing through line-of-sight and "hearing." The ultimate objective is to secure more pills on one's side by the end of the time limit.

This problem is important because it encapsulates many of the key challenges in modern multi-agent systems and autonomous decision-making. Agents must operate under partial observability, reason about both cooperative and adversarial behavior, and adapt to dynamic, unpredictable environments. Additionally, the need for fast, strategic decision-making under uncertainty mirrors real-world applications such as robotic team coordination, autonomous surveillance, and adversarial planning. Successfully designing agents to compete effectively in this environment requires addressing critical topics in artificial intelligence, including behavior modeling, reinforcement learning, real-time strategy switching, and robust multi-agent communication under noisy conditions.

1.1 Contribution

This work presents a system designed to navigate through a PacMac capture the flag game and perform actions according to various game states. The main contributions are the following:

- A custom behavior tree to allow each agent on the team to decide its current action based on various conditions.
- RL models for each of the three actions including attack, defend, and escape.

1.2 Outline

This paper is structured as follows. Section 2 reviews related work on hierarchical decision-making with behavior trees, reinforcement learning techniques, and multi-agent coordination strategies, with particular focus on

methods that influenced our system design. Section 3 describes our implemented approach in detail, outlining the structure of the custom behavior tree, the training and configuration of the three specialized reinforcement learning models, and how these components were integrated to manage agent behavior in real time. Section 4 presents the experimental setup and the evaluation of our system's performance, including comparisons with other teams, analysis of results, and identification of key shortcomings. Section 5 discusses the technical and managerial aspects that contributed to the limitations of our approach, reflecting on lessons learned during development. Finally, Section 6 concludes the paper by summarizing our contributions and proposing directions for future work to improve adaptability and robustness in multi-agent systems using reinforcement learning and behavior trees.

2 Related work

Autonomous decision-making in competitive, multi-agent environments has been a long-standing focus in artificial intelligence and robotics, with applications ranging from robotic sports and strategy games to real-world security and search-and-rescue missions. This project builds on established techniques in behavior trees and reinforcement learning, adapting them to a real-time, adversarial setting within a 3D Unity implementation of PacMan Capture the Flag. In this section, we review prior work on hierarchical control architectures, learning-based behavior selection, and multi-agent coordination under partial observability, highlighting how our approach extends and contrasts with these methods.

A foundational reference for the reinforcement learning techniques used in this project is the textbook by Sutton and Barto [5]. Their work introduces the core concepts of RL, such as the agent-environment interaction model, cumulative reward maximization, and policy optimization, all of which underpin the training of the models in our system. Techniques such as temporal-difference learning and policy gradient methods described in their book form the theoretical basis for the models we implemented using Unity's ML-Agents toolkit. In particular, the focus on how agents learn optimal behaviors through trial-and-error interactions with an environment directly influenced the design of our attack, defense, and escape policies.

A fundamental part of our system is the use of separate reinforcement learning (RL) models for different agent behaviors, a component inspired by Wagner [6] in his degree paper. Wagner applied this technique to modularize control in a robotic setting, using distinct RL policies for different subtasks and switching between them as needed. Additionally, he paired these mod-

els with a behavior tree structure to manage high-level decision-making and ensure stability by constraining learning agents within predefined subgoal boundaries. This combination of modular RL and structured behavior control influenced our design, leading us to adopt a similar approach where specialized models are selected dynamically through a custom behavior tree based on the agent's situation.

Other works have also demonstrated the effectiveness of Unity's ML-Agents framework for complex agent training. Dandoti [1] explored the use of ML-Agents to train agents for survival scenarios, where agents learned to gather resources while avoiding threats. Although the survival context differs from our capture-the-flag environment, the underlying methodology of using ML-Agents to model adaptive behavior closely aligns with our approach. Dandoti's work reinforced the importance of careful reward design and environment setup when training agents to perform specialized tasks.

Similarly, Savid et al. [3] investigated reinforcement learning-based training of autonomous driving agents in Unity environments. By comparing the performance of different RL algorithms, notably Proximal Policy Optimization (PPO) and POCA, they demonstrated the effectiveness of Unity ML-Agents in dynamic, obstacle-rich environments. Their exploration of behavioral cloning for pre-training agents also informed our understanding of how agent generalization could be enhanced through smarter training processes, particularly in unpredictable environments like ours.

Finally, an important recent advancement in combining behavior trees and reinforcement learning is presented by Li et al. [2], who propose the MARL-BT framework. Their system integrates multi-agent reinforcement learning (MARL) directly into behavior trees, allowing agents to continue learning during execution without needing separately trained models. They also introduce an action masking technique to handle conflicts between rule-based and learning-based tasks. While their work represents a more advanced integration of learning and decision-making than ours, our system shares a similar motivation: using behavior trees to modularize complex agent behavior and applying reinforcement learning models to specialized sub-tasks. In contrast to MARL-BT, our approach relies on selecting among pre-trained RL models based on the game state, offering a simpler method for dynamic decision-making in a competitive multi-agent environment. In future work, it would be interesting to explore how integrating live learning mechanisms like MARL-BT could further improve adaptability and coordination in our system.

3 Proposed method

Our implemented system is built around a custom behavior tree architecture inspired by the work of Wagner [6]. This behavior tree enables each agent to dynamically select between three high-level strategies—attack, defend, or escape—based on the current state of the game. Each strategy corresponds to a dedicated model trained using Unity's ML-Agents toolkit. These models are specialized for their respective roles: the attack model focuses on collecting pills in enemy territory, the defend model targets opponents who attempt to steal pills, and the escape model guides agents back to safety when under threat. This modular approach allows for flexible, state-driven behavior while maintaining the benefits of reinforcement learning within well-defined action domains.

3.1 Behavior Tree

Our behavior tree is made up of multiple classes including a sequence, a selector, a switched learning action, a condition, and a predicate condition class. It is composed of any nested combination of these classes. Before describing the structure of the tree we used, it is best to define each of these classes and explain their function within the behavior tree.

- **Sequence** – A composite node that executes its child nodes in order until one of them fails or returns running. If all child nodes succeed, the sequence returns success. It is used when all conditions and actions in the sequence must be satisfied for the behavior to succeed.
- **Selector** – A composite node that executes its child nodes in order until one of them succeeds or returns running. If all child nodes fail, the selector returns failure. It is used when only one successful branch is needed to proceed.
- **Switched Learning Action** – A leaf node responsible for switching the agent's brain to a specific trained policy (e.g., attack, defend, escape) based on the game state.
- **Condition** – A node that evaluates a boolean expression about the current environment or agent state. It returns success or failure depending on whether the condition is met.
- **Predicate Condition Node** – A specific type of condition node that wraps a custom predicate (boolean function) and evaluates it at runtime. It is used to guide branching decisions within the tree.

The root of our tree is a sequence node, as we want each agent to first check whether defense is needed before proceeding to attack the opponent. The condition used to decide when to defend checks whether there is an enemy agent carrying food pills and whether a teammate is already assigned to defense. If no agent is currently defending and an enemy is carrying pills, the agent selects the defense action.

If defense is not required, the agent proceeds to the next selector node, which determines whether to attack or escape. This selector begins with a predicate condition node that checks if there is food available; if so, the agent should attempt to attack. However, the attack action is not invoked directly after this condition, as we only want the agent to attack if it is safe to do so. To implement this logic, we include another selector within a sequence node. This selector contains a safety condition, which verifies that the agent is currently a PacMan and that no enemy agents are within a 5f radius. If the safety condition fails, the agent falls back to the escape action; if it passes, the agent proceeds to attack using the corresponding switched learning action.

In summary, the behavior tree design allows each agent to make informed decisions based on the current game state, prioritizing defense when necessary, and dynamically choosing between escape and attack based on safety conditions. This modular structure enables agents to react adaptively while maintaining a clear hierarchy of priorities.

3.2 Reinforcement Learning

Reinforcement learning (RL) helps agents learn optimal policies by maximizing cumulative rewards through trial-and-error interactions [4]. We adopted RL to address the dynamicand mmulti-agentnature of Pacman CTF, where rule-based strategies alone fail to adapt to adversarial tactics. Our hybrid architecture combines RL with behavior trees (Section 3.1), enabling agents to learn specialized behaviors while maintaining strategic coherence.

3.2.1 Design of Agent Policies

The complexity of Pacman CTF necessitated decomposing agent behavior into three specialized roles: attack, defend, and escape. These roles were designed to be mutually exclusive and exhaustive, ensuring comprehensive coverage of in-game scenarios. The attack policy focuses on navigating to enemy territory, collecting pills, and returning safely. The defend policy prioritizes intercepting opponents carrying pills, while the escape policy triggers retreats to avoid elimination when threats are detected.

Training occurred in two phases to balance specialization and adaptability. In the first phase, each policy was pretrained in isolation within simplified environments (e.g., symmetric maps with a single agent). This allowed foundational skills to develop without interference from competing objectives. During the second phase, policies were fine-tuned jointly in competitive scenarios involving multiple agents and asymmetric maps. This phased approach ensured robust performance in complex environments while mitigating the risk of reward signal dilution.

A composite episode framework was implemented to manage transitions between policies. Each composite episode comprises multiple local episodes, where a local episode corresponds to the execution of a single behavior (e.g., attacking). Termination conditions for local episodes include postcondition fulfillment, or timeout. Composite episodes terminate after 10,000 steps or when all goal conditions are met, ensuring agents do not stagnate in unproductive states.

3.2.2 Rewards

Rewards were carefully structured to align with role-specific objectives while maintaining training stability (Table 1). Attack agents receive a reward of +0.0125 for collecting enemy pills, incentivizing pill retrieval, but incur a penalty of -1 for elimination to discourage reckless behavior. Defend agents earn +1 for eliminating opponents, promoting proactive defense, while escape agents face a harsher penalty of -2 for elimination to prioritize survival. A nominal time penalty of -0.00002 per step was applied universally to encourage urgency.

To prevent training instability, all rewards were normalized to the range [-1, +1], as recommended by Unity's ML-Agents toolkit. This normalization ensured consistent gradient updates during policy optimization. Reward components were deliberately simplified to avoid overfitting; for example, obstacle collision penalties were omitted for attack and defend policies to avoid excessive risk aversion.

Name	**Attack**	**Defend**	**Escape**
Post heightTime Penalty	-0.00002	-0.00002	-0.00002
Eat Enemy Pill	+0.0125	-	-
Kill Enemy	-	+1	-
Die to Enemy	-1	-1	-1 + -0.1*carried pills
Score	-	-	-

Table 1: rewards

3.2.3 Observations

Observations were tailored to each role to reduce input dimensionality and accelerate learning (Table 2). Attack agents receive positions of enemy pills and raycast data to navigate around obstacles, while defend agents track the number of active defenders and positions of pill-carrying opponents. Escape agents prioritize proximity to threats, monitoring distances to visible enemies.

A blackboard script was used to track the position, current action, carried pills of friendly agents and positions of enemy position. Spatial observations (e.g., positions, distances) were normalized to [-1, +1] to stabilize neural network inputs. Raycast sensors provided localized obstacle detection through six directional beams, each returning distances to walls or agents. Null observations, such as absent pills, were zero-padded to maintain fixed input sizes across varying game states.

Name	**Attack**	**Defend**	**Escape**
Position of Agent	X	X	X
Agent's Team	X	X	X
Distance to Visible Enemies	X	X	X
Distance to Friendly Agents	X	X	-
Score	X	-	-
Position of Friendly Pills (x3)	-	X	-
Position of Enemy Pills (x3)	X	-	-
Obstacle Raycast (x6)	X	X	X
CarriedPills of Agent	X	-	X
Carried Pills of FriendlyAgents	-	-	-
Carried Pills of Enemy	-	X	-
Ghost State	X	X	X
No. of Friendly Defenders	-	X	-
No. of Friendly Attackers	X	-	-

Table 2: Observations of RL Agents of Different Behaviors, x indicates that the observation is recieved and - indicates otherwise

3.2.4 Actions

All of the RL agents share the same discrete action space. This action space consists of 25 actions inspired from [6] Those actions are organized in a 5x5 grid. Discrete actions were favored over continuous controls to reduce the risk of erratic behavior during policy initialization.

3.3 Configurations

Proximal Policy Optimization (PPO) [4] was selected for its balance of sample efficiency and stability in multiagent environments. Key hyperparameters included a learning rate of 3e4 (decaying by 10% every 50,000 steps) and a batch size of 1,024 steps per policy update. To enhance robustness, self-play was integrated, periodically replacing adversaries with historical agent versions. Curriculum learning further improved adaptability by incrementally increasing environmental complexity—training began in small symmetric maps and progressed to tournament-sized asymmetric layouts.

4 Experimental results

This section evaluates the performance of our RL-based agents in the Pacman CTF tournament, comparing results with other teams and analyzing the effectiveness of competing strategies.

4.1 Experimental setup

The tournament, and thereby the test of our agent, involved 18 teams competing in 2 Maps (a simple map D and a bigger complex map B). The teams were judged based on the number of games won and the score difference. Teams were allowed one week between sessions to refine strategies based on initial opponent analysis.

4.2 Analysis of Outcome

	Map B	Map D
Group 1	39	46
Group 8	33	37
Group 7	1	6
Group 13	0	1

Table 3: Table comparing the total points by different teams

As shown in Table 3, our overall performance in the competition was unsatisfactory. While our initial results—developed using a single agent in a

simplified environment—were promising, we were unable to produce a functional model for the official competition maps. A detailed breakdown of the issues encountered is provided in the failure analysis section 5.

Despite these challenges, our team did achieve partial success during the finals. Specifically, in two games, one of our agents reliably executed an attack strategy targeting a specific pill and appropriately switched to an escape policy. However, due to the opponent's effective defensive behavior, the agent failed to return the pills, resulting in either a loss or, at best, a draw.

Among the different planning strategies tested, decision trees and behavior trees emerged as the most effective for orchestrating agent actions. Although our final results were not satisfactory, we maintain that our system architecture—particularly the behavior tree design—was conceptually sound and offered a robust foundation.

Interestingly, nearly all teams adopted a similar high-level strategy: develop discrete modules for attack and defense, and integrate them using a behavior tree or state machine. Some teams also emphasized border defense as a core component of their strategy.

Comparative Analysis with Other Teams: Top-Performing Teams (Group 1 and Group 8) utilized behavior trees for high-level decision making, supplemented by fine-tuned classical path planning techniques such as A*, Dijkstra maps, and velocity obstacles for reliable navigation and opponent avoidance. Group 1 employed a more sophisticated behavior tree but encountered issues with local minima, possibly due to the absence of backward chaining mechanisms as discussed in [7], which prioritize goal-driven behavior over reactive transitions. Group 8 opted for a simpler tree but achieved comparable success due to effective tuning of their path planning modules. Group 7 implemented reinforcement learning (RL) to select among four predefined action modes: Escaping, Catching, Guarding, and Collecting. Their approach was conceptually strong, allowing RL to discover non-obvious strategies that might be missed in manually crafted trees. However, they faced convergence issues, with agents frequently oscillating between behaviors, ultimately stalling task progress.

Our Team's Hybrid Approach: Among the four teams that used reinforcement learning, we were the only team to integrate it with a behavior tree framework. While our hybrid model only worked in a simplified environment and failed on the full competition maps, we successfully demonstrated its feasibility. The process exposed us to a range of state-of-the-art ideas, and we believe—with further tuning and more development time—our system has the potential to perform at a championship level.

Overall, Behavior trees clearly demonstrated their effectiveness as a con-

trol architecture across most top-performing teams. While our implementation remains rudimentary, it is grounded in goal-oriented planning principles and has shown promise under constrained conditions.

5 Analysis of Failure

Our method failed to win any competition. Despite successfully demonstrating basic attack and defense behaviors in small, single-agent scenarios, our system could not scale to the full multi-agent environment. In the following sections, we will discuss the technicalities of our failure while identifying the core issue and possible solutions for it.

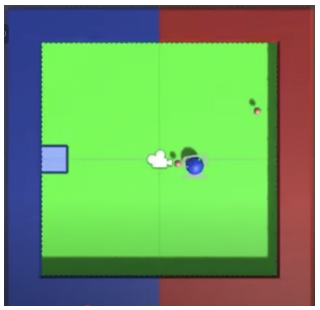
5.1 Technical Aspect of Failure

Our experimental pipeline was designed to incrementally build complexity: first training an attack policy in an empty arena, then progressively introducing obstacles until the competition map was fully replicated, followed by defense and escape policies seeded from the converged attack model. We hypothesized that a robust attack policy would bootstrap the entire stack, yet the attack stage never converged to anything beyond a near-random walk.

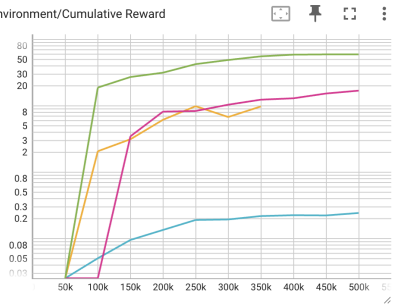
In single-agent small environments, agents successfully learned basic pill collection. Multiagent coordination without obstacles with homogeneous teams also showed promise. However, scaling to adversarial multiagent scenarios or obstacle-dense tournament maps led to very erratic training behavior, which we were never able to fix. The observation space grew exponentially with the number of agents and map complexity, leading to noisy inputs that hindered policy convergence.

Despite exhaustive reward tuning—including group-based rewards, flipped observation/action encoding, and self-play regimes—we observed persistent collapse of meaningful behaviors. As the number of agents grew and the map expanded, the dimensionality of our raw observation vector exploded, leading to severe sample inefficiency. We were not able to reason some behaviours like Figure 2b. Our attempt to discretize spatial observations via a Dijkstra-grid, increased the computation time exponentially, but with more time we could've explored more on making this approach more efficient. As a fallback, we also integrated an A* planner into the BT that would execute a simple path-finding to move the agent to the enemy side before attacking. This fallback produced only marginal benefit and could not overcome the underlying instability of the learned policy.

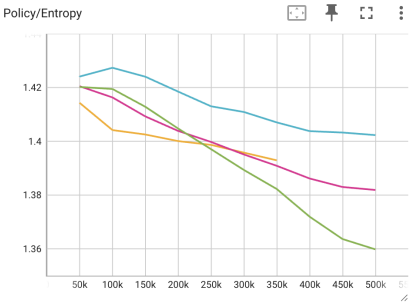
Throughout training, we leveraged ML-Agents' TensorBoard hooks to



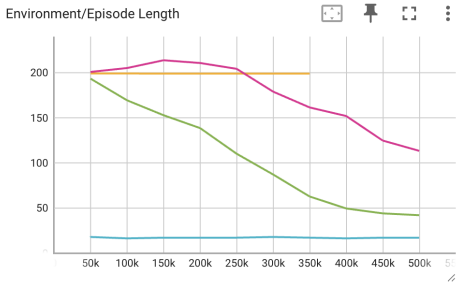
(a) A small map with 2 pills



(b)

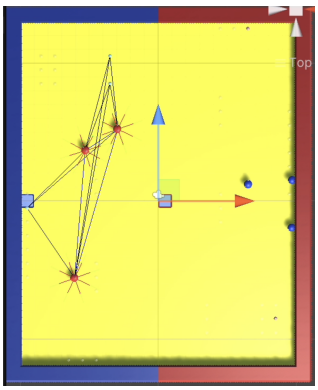


(c)

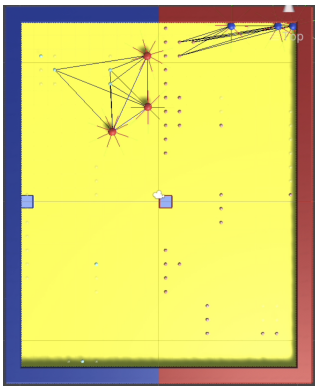


(d)

Figure 1: Successful training in simpler environment



(a) A successful episode, where the attack policy collects all the pills



(b) At a latter instance during the same training, all the blue agents tend to corner towards a specific corner

Figure 2: Screenshots from training of Attack policy in Map D without obstacles

monitor entropy, reward trajectories, and loss curves in real time, and relied on periodic model checkpoints to detect overfitting or exploration collapse. These diagnostics provided clear insight into what was going wrong but proved insufficient in guiding how to retune reward functions or observation embeddings effectively. More advanced ideas—such as particle filtering to infer unseen opponent positions or capsule-based relational modules—remained on our backlog, as we lacked the time to prototype and integrate them.

5.2 Managerial Aspect of Failure

On the organizational side, our use of Notion for goal tracking, milestone planning, and task assignment was sound, and we held regular reviews that kept the team aligned. In the first week, we completed the literature review and ML-Agents sandbox tests; in the second, we parallelized BT integration with initial attack-policy training; in the third, we devoted ourselves to reward and observation tuning. By the time we realized the risk of non-convergence, there was insufficient time to engineer robust rule-based or curriculum frameworks. We took a deliberate decision to fail wholeheartedly by trying our best to improve our proposed approach and learn better during the process rather than dedicating the final days to the partial integration of a curriculum strategy that would have yielded only provisional and underdeveloped solutions.

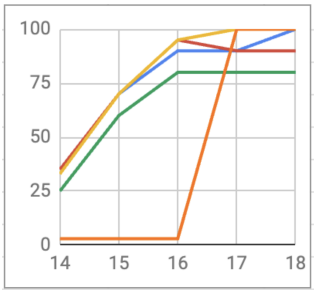


Figure 3: Progress Chart

6 Summary and Conclusions

In this work, we introduced a hybrid control architecture for a 3D Unity "PacMan Capture the Flag" scenario, combining a custom behavior tree (BT) with three role-specific reinforcement-learning (RL) policies—attack, defend, and escape. The behavior tree encodes simple, prioritized predicates

(e.g., whether an opponent carries pills or if the local vicinity is safe) to select among pretrained PPO agents, thereby blending strategy switching with learned motion control. Each RL policy was trained in two phases—isolated map drills followed by adversarial multi-agent fine-tuning—and employs observations and rewards tailored to its objective (pill collection, interception, or retreat), normalized to stabilize learning.

Although the approach performed well in single-agent tests and simplified arenas, it did not scale to the full competition settings. In an 18-team tournament across two maps, our agents failed to secure a win and registered only one scoring event. We found that as the number of agents and maze complexity grew, the high-dimensional observation vectors introduced noisy inputs that impeded policy convergence, and sparse, role-specific rewards proved difficult to tune across heterogeneous interactions.

These results highlight the tension between pure RL methods and the demands of large-scale adversarial environments. While behavior trees provided a clear decision hierarchy, they could not compensate for the sample inefficiency and brittleness of the underlying policies.

References

[1] Sarosh Dandoti. Learning to survive using reinforcement learning with ml-agents. *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 10(VII):3009–3014, 2022.

[2] Xianglong Li, Yuan Li, Jieyuan Zhang, Xinhai Xu, and Donghong Liu. Embedding multi-agent reinforcement learning into behavior trees with unexpected interruptions. *Complex & Intelligent Systems*, 2024.

[3] Yusef Savid, Reza Mahmoudi, Rytis Maskeliūnas, and Robertas Damaševičius. Simulated autonomous driving using reinforcement learning: A comparative study on unity's ml-agents framework. *Information*, 14(5):290, 2023.

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition edition, 2018.

[6] Jannik Wagner. Improving behavior trees that use reinforcement learning with control barrier functions: Modular, learned, and converging control

through constraining a learning agent to uphold previously achieved sub goals. Degree project in computer science, second cycle, KTH Royal Institute of Technology, 2025.

[7] Petter Ögren. Convergence analysis of hybrid control systems in the form of backward chained behavior trees. *IEEE Robotics and Automation Letters*, 5(4):6073–6080, 2020.